SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

TECHNICAL PRESENTATION

1. Introduction to Compilers

a) A compiler translates high-level language into machine code.
b) It improves execution efficiency.
c) Performs syntax and semantic checks.
d) Produces optimized output code.
e) Essential in software development.

2. Language Processors Overview

a) Includes compiler, interpreter, and assembler.
b) Each processes source code differently.
c) Compilers generate executable files.
d) Interpreters execute line by line.
e) Assemblers convert assembly to machine code.

3. Compiler vs Interpreter

a) Compiler translates entire program at once.
b) Interpreter processes line-by-line.
c) Compiled code runs faster.
d) Interpreted code is easier to debug.
e) Used in different programming scenarios.

4. Phases of a Compiler

a) Compiler has multiple phases.
b) Each phase performs a specific task.
c) Includes lexical, syntax, semantic analysis.
d) Also includes optimization and code generation.
e) Works sequentially.

a.

5. Lexical Analysis Phase

a) First phase of compilation.

b) Converts input into tokens.
c) Removes whitespace and comments.
d) Uses finite automata.
e) Reports lexical errors.

6. Syntax Analysis (Parsing)

a) Checks grammatical structure.
b) Builds parse tree.
c) Uses CFG rules.
d) Detects syntax errors.
e) Uses parsers like LL and LR.

a.

7. Semantic Analysis

a) Ensures meaning correctness.
b) Checks types and scope.
c) Uses symbol table.
d) Detects semantic errors.
e) Prepares for intermediate code.

8. Intermediate Code Generation

a) Generates machine-independent code.
b) Example: three-address code.
c) Simplifies optimization.
d) Bridges front-end and back-end.
e) Improves portability.

9. Code Optimization

a) Improves code efficiency.
b) Reduces execution time.
c) Eliminates redundancy.
d) Uses techniques like loop optimization.
e) Makes code faster.

10. Code Generation

a) Produces target machine code.

b) Uses registers and memory.
c) Considers architecture.
d) Final phase of compiler.
e) Outputs executable program.

11. Symbol Table Management

a) Stores variable information.
b) Tracks scope and type.
c) Used across all phases.
d) Supports error checking.
e) Efficient access is important.

12. Error Handling in Compilers

a) Detects and reports errors.
b) Types: lexical, syntax, semantic.
c) Uses recovery techniques.
a) Improves debugging.
b) Important for user-friendly compilers.

13. Grouping of Compiler Phases

a) Front-end and back-end division.
b) Front-end handles analysis.
c) Back-end handles synthesis.
d) Improves modular design.
e) Simplifies implementation.

14. Front-End of Compiler

a) Includes lexical, syntax, semantic phases.
b) Generates intermediate code.
c) Machine-independent.
d) Focuses on correctness.
e) Easier to modify.

15. Back-End of Compiler

a) Includes optimization and code generation.

b) Machine-dependent.
c) Produces final output code.
d) Focuses on performance.
e) Works on intermediate code.

16. Compiler Construction Tools

a) Tools to build compilers easily.
b) Examples: LEX, YACC.
c) Automates phases.
d) Saves development time.
e) Widely used in industry.

17. Introduction to LEX Tool

a) LEX is a lexical analyzer generator.
b) Converts patterns into code.
c) Uses regular expressions.
d) Generates C code.
e) Simplifies token recognition.

18. Structure of a LEX Program

a) Divided into three parts.
b) Definitions section.
c) Rules section.
d) User code section.
e) Each has specific purpose.

19. Role of Lexical Analyzer

a) Scans source code.
b) Converts into tokens.
c) Removes unnecessary characters.
d) Passes tokens to parser.
e) Handles lexical errors.

20. Tokens and Lexemes

a) Token: category of symbols.

b) Lexeme: actual string.
c) Example: id, number.
d) Essential in parsing.
e) Defined using patterns.

21. Specification of Tokens

a) Uses regular expressions.
b) Defines valid patterns.
c) Helps lexical analysis.
d) Ensures correctness.
e) Easy to implement.

22. Regular Expressions in Compilers

a) Describe token patterns.
b) Used in lexical analysis.
c) Simple and powerful.
d) Converted to automata.
e) Widely used.

23. Finite Automata in Lexical Analysis

a) Used for token recognition.
b) DFA and NFA types.
c) Efficient processing.
d) Converts regex to automata.
e) Foundation of lexical analyzers.

24. NFA vs DFA

a) NFA has multiple transitions.
b) DFA has single transition.
c) DFA is faster.
d) NFA is easier to design.
e) Conversion is possible.

25. Transition Diagrams

a) Graphical representation.

b) Shows state transitions.
c) Used in lexical analysis.
d) Helps understanding automata.
e) Easy visualization.

26. Input Buffering Techniques

a) Improves reading efficiency.
b) Uses buffers.
c) Handles large input.
d) Reduces I/O operations.
e) Important for performance.

27. Lexical Errors

a) Occur in token formation.
b) Example: invalid symbols.
c) Detected in first phase.
d) Error recovery needed.
e) Improves program correctness.

28. Bootstrapping in Compilers

a) Compiler written in same language.
b) Self-hosting concept.
c) Improves portability.
d) Used in advanced compilers.
e) Complex but powerful.

29. Cross Compiler

a) Runs on one machine.
b) Generates code for another.
c) Used in embedded systems.
d) Supports portability.
e) Widely used in development.

30. One-Pass vs Multi-Pass Compiler

a) One-pass: faster, less memory.

b) Multi-pass: more accurate.
c) Used for complex languages.
d) Trade-off exists.
e) Depends on requirements.

31. Syntax-Directed Translation

a) Combines parsing and translation.
b) Uses attributes.
c) Generates intermediate code.
d) Based on grammar rules.
e) Important in compilers.

32. Attribute Grammars

a) Extends CFG.
b) Adds semantic information.
c) Uses synthesized attributes.
d) Used in semantic analysis.
e) Improves translation.

33. Parsing Techniques Overview

a) Top-down and bottom-up.
b) LL and LR parsing.
c) Used in syntax analysis.
d) Each has advantages.
e) Widely used methods.

34. Top-Down Parsing

a) Starts from root.
b) Builds parse tree.
c) Uses LL parser.
d) Simple implementation.
e) May require backtracking.

35. Bottom-Up Parsing

a) Starts from input.

b) Builds parse tree upward.
c) Uses shift-reduce method.
d) More powerful.
e) Handles complex grammars.

36. Role of YACC Tool

a) Parser generator tool.
b) Works with LEX.
c) Generates syntax analyzer.
d) Uses grammar rules.
e) Simplifies parser design.

37. Compiler Design Challenges

a) Handling errors.
b) Optimization complexity.
c) Memory management.
d) Machine dependency.
e) Efficiency concerns.

38. Applications of Compilers

a) Software development.
b) Embedded systems.
c) Game engines.
d) AI programming.
e) System software.

39. Evolution of Compilers

a) From simple translators.
b) To modern optimizing compilers.
c) Supports multiple languages.
d) Improves performance.
e) Continues evolving.

40. Future Trends in Compiler Design

a) AI-based optimization.

b) Just-in-time compilation.
c) Parallel compilation.
d) Cloud-based compilers.
e) Increasing automation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand